PHASE 2 · COMPUTER SCIENCE FUNDAMENTALS

5. DBMS

Study Handnote & Practice Workbook

Normalization • Transactions • ACID • Indexes • Query Optimization • Joins • Locks • Isolation Levels • Sharding • Replication • CAP Theorem

MySQL / InnoDB first | PostgreSQL differences flagged
Includes cheat sheet, 12-exercise lab, 30 interview questions, full answers

Phase 2 — CS Fundamentals · 5. DBMS

Study handnote — Normalization → CAP Theorem Examples use MySQL/InnoDB by default (PostgreSQL differences called out). Schema examples are modelled on a bus/ship ticketing domain so they map to real work.

How to use this file

1. Read a section → close the file → write the *Recall card* from memory.
2. Run the SQL in the **Lab** section (§13) against a scratch DB.
3. Answer the **Practice** questions at the end of each section before checking §15.

Table of contents

1. Normalization

1.1 The problem it solves

Un-normalized tables cause three **anomalies**:

Anomaly	Meaning	Example
Insert	Can't record fact A without knowing fact B	Can't add a new bus operator until they have at least one trip
Update	Same fact stored many times → partial updates → contradictions	Operator phone stored on 50k tickets; update misses 3 rows
Delete	Removing a row destroys unrelated information	Deleting the last trip of an operator erases the operator entirely

Root cause: **one table is storing facts about more than one thing.**

1.2 Functional dependency (FD)

`X → Y` — "X determines Y": for any two rows with the same X, Y must be equal.

```
ticket_id       → seat_no, price, trip_id     (ticket_id determines everything about the ticket)
trip_id         → route_id, departure_time
(trip_id, seat_no) → ticket_id                 (a seat on a trip is sold once)
```

Types you must be able to name:

- **Full FD** — Y depends on the *whole* composite key, not part of it.
- **Partial FD** — Y depends on *part* of a composite key. → breaks 2NF.
- **Transitive FD** — `X → Y`, `Y → Z`, so `X → Z` through a non-key attribute. → breaks 3NF.
- **Trivial FD** — `X → Y` where Y ⊆ X. Always true, ignore.

Key definitions

- *Super key*: any attribute set that determines the whole row.
- *Candidate key*: minimal super key (remove any attribute and it stops determining).
- *Primary key*: the candidate key you picked.
- *Prime attribute*: an attribute that is part of **some** candidate key.

1.3 The normal forms

1NF — atomic values

- No repeating groups, no arrays/CSV in a column, no multi-valued cells.
- Every row is unique (has a key).

✗ Bad

```
| booking_id | passenger_names            | seats     |
| 101        | "Rahim, Karim, Salma"      | "A1,A2,A3"|
```

✓ Good — child table `booking_passengers(booking_id, passenger_name, seat_no)`.

> Note: a **JSON column** is a deliberate, pragmatic 1NF violation. Fine for schemaless payload (gateway response, audit diff), bad for anything you filter/join on.

2NF — no partial dependency

Applies only when the PK is **composite**. Every non-prime attribute must depend on the *entire* key.

✗ Bad — PK is `(trip_id, seat_no)`

```
trip_seat(trip_id, seat_no, seat_class, bus_registration_no, departure_time)
                                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

   bus_registration_no and departure_time depend only on trip_id → partial FD
```

✓ Split: `trip(trip_id, bus_registration_no, departure_time)` + `trip_seat(trip_id, seat_no, seat_class)`.

3NF — no transitive dependency

Non-prime attributes must depend on the key, **the whole key, and nothing but the key**.

✗ Bad

```
trip(trip_id, route_id, route_name, origin_city, destination_city)
     trip_id → route_id → route_name        (transitive)
```

✓ Split: `trip(trip_id, route_id, ...)` + `route(route_id, route_name, origin_city, destination_city)`.

BCNF — stricter 3NF

For every non-trivial FD `X → Y`, **X must be a super key**.

3NF allows an exception: if Y is a *prime* attribute, a transitive dependency is tolerated. BCNF removes that exception.

Classic failing case:

```
counter_assignment(counter_id, service_type, agent_id)
FDs: (counter_id, service_type) → agent_id       &#10003; key
     agent_id → service_type                     &#10007; agent_id is not a super key
```

This is in 3NF (`service_type` is prime) but **not** BCNF. Decompose into `agent_service(agent_id, service_type)` + `counter_agent(counter_id, agent_id)`.

> Trade-off: BCNF decomposition is always lossless but **not always dependency-preserving**. 3NF is always both. That's why 3NF is the practical target.

4NF — no multi-valued dependency (MVD)

An MVD `X &#8608; Y` means: for a given X, the set of Y values is independent of the other attributes.

✗ Bad — an operator serves multiple routes AND accepts multiple payment methods (independent facts)

```
| operator_id | route   | payment_method |
| GL          | DHK-CTG | bKash          |
| GL          | DHK-CTG | Nagad          |
| GL          | DHK-SYL | bKash          |
| GL          | DHK-SYL | Nagad          |   ← cartesian explosion
```

✓ Two tables: `operator_route`, `operator_payment_method`.

5NF / PJNF — no join dependency

Table can't be losslessly decomposed further. Rare in practice; mention it, don't chase it.

6NF

One non-key attribute per table. Used in temporal / anchor-modelling data warehouses only.

1.4 Decomposition properties

- **Lossless join**: `R1 ⋈ R2 = R`. Guaranteed if the shared attributes form a key of at least one side. Non-negotiable.
- **Dependency preservation**: all original FDs are still enforceable without a join. Nice to have.

1.5 Denormalization — deliberate, measured

Normalize first. Denormalize only with a measured reason.

Technique	Use case	Cost
Redundant column (`ticket.route_name`)	Kill a join on a hot read path	Must sync on update
Counter cache (`trip.sold_seats_count`)	Avoid `COUNT(*)` per request	Race conditions → update atomically or in a transaction
Summary/rollup table (`daily_sales`)	Reporting	Staleness, rebuild job
Materialized view	Complex aggregate	Refresh strategy (PG has them natively; MySQL needs a table + job)
Historical snapshot (`ticket.price_at_purchase`)	**Not** denormalization — it's a different fact	None, this is correct design

⚠ That last row matters: storing the price on the ticket is *right*, because "price charged" and "current fare" are genuinely different facts. Don't "normalize" away history.

1.6 Recall card

> 1NF atomic · 2NF no partial FD · 3NF no transitive FD · BCNF every determinant is a super key · 4NF no independent MVD · 5NF no join dependency. Mnemonic: "*the key, the whole key, and nothing but the key — so help me Codd*."

1.7 Practice

1. Given `orders(order_id, product_id, qty, product_name, unit_price, customer_id, customer_email)` with PK `(order_id, product_id)` — identify every violation and produce a 3NF schema.
2. Why is every BCNF relation also in 3NF, but not vice versa?
3. Give a case where you'd intentionally stay at 2NF.
4. `R(A,B,C,D)` with `A→B`, `B→C`, `D→A`. What are the candidate keys? Highest normal form?

2. Transactions

2.1 Definition

A **transaction** is a unit of work that moves the database from one consistent state to another, and is *all-or-nothing*.

```
START TRANSACTION;                    -- or BEGIN
  UPDATE trip_seat SET status='sold' WHERE trip_id=91 AND seat_no='A1';
  INSERT INTO ticket (trip_id, seat_no, price, user_id) VALUES (91,'A1',1200,7);
  UPDATE wallet SET balance = balance - 1200 WHERE user_id = 7;
COMMIT;                               -- or ROLLBACK
```

2.2 Transaction states

```
        begin            end of ops        flush to disk
Active ──────→ Partially Committed ──────→ Committed
   │                     │
   │ error               │ error
   ▼                     ▼
 Failed ──rollback──→ Aborted (→ restart or kill)
```

2.3 Autocommit

- MySQL: `autocommit = 1` by default → **every statement is its own transaction**.
- Explicit `START TRANSACTION` suspends autocommit until COMMIT/ROLLBACK.
- Check: `SELECT @@autocommit;`
- DDL (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`) causes an **implicit commit** in MySQL — you cannot roll back a migration statement. PostgreSQL *does* support transactional DDL. This is a genuinely important difference for migrations.

2.4 Savepoints — partial rollback

```
START TRANSACTION;
  INSERT INTO booking ...;
  SAVEPOINT after_booking;
  INSERT INTO payment_attempt ...;      -- may fail on gateway timeout
ROLLBACK TO SAVEPOINT after_booking;    -- keeps the booking, drops the attempt
COMMIT;
```

Laravel nests transactions using savepoints automatically — a nested `DB::transaction()` does **not** start a new real transaction, it creates `SAVEPOINT trans2`. Important consequence: an inner "commit" isn't durable until the outermost one commits.

2.5 In Laravel

```
// Auto commit/rollback + deadlock retry (5 attempts)
DB::transaction(function () use ($request) {
    $seat = TripSeat::where('trip_id', $request->trip_id)
        ->where('seat_no', $request->seat_no)
        ->lockForUpdate()          // SELECT ... FOR UPDATE
        ->firstOrFail();

    abort_if($seat->status !== 'available', 409, 'Seat already taken');

    $seat->update(['status' => 'sold']);
    Ticket::create([...]);
}, 5);

// Manual control
DB::beginTransaction();
try { ...; DB::commit(); }
catch (\Throwable $e) { DB::rollBack(); throw $e; }
```

Traps

- Dispatching a queue job inside a transaction → the worker may pick it up **before** commit and read stale/absent data. Fix: `afterCommit()` on the job, or `'after_commit' => true` in `config/queue.php`.
- Sending an email / calling a payment gateway inside a transaction → external side effect can't be rolled back, and it holds locks for the whole HTTP round-trip. Do I/O outside.
- Long transactions in MySQL keep the **undo log history** alive → bloats the tablespace and slows every other reader. Keep them short.

2.6 Recall card

> Transaction = atomic unit. States: Active → Partially Committed → Committed / Failed → Aborted. MySQL autocommits per statement and implicitly commits DDL; PostgreSQL has transactional DDL. Savepoints give partial rollback (Laravel nesting = savepoints). Never do network I/O inside a transaction.

2.7 Practice

1. Why does Laravel's nested `DB::transaction()` not create a real nested transaction?
2. You run a migration that fails halfway on MySQL. What state is the schema in, and why?
3. What breaks if you `Mail::send()` inside a transaction that later rolls back?

3. ACID

3.1 The four properties

A — Atomicity

All statements in a transaction succeed, or none apply.

How InnoDB does it: the **undo log**. Before modifying a page, the old version of the row is written to the undo log. `ROLLBACK` replays the undo log backwards. A crash mid-transaction is resolved at startup by rolling back uncommitted transactions.

The undo log serves double duty — it's also how MVCC builds old row versions for readers (§8).

C — Consistency

The database moves from one valid state to another, respecting all defined rules:

- Primary/unique/foreign keys
- `CHECK` constraints, `NOT NULL`, column types
- Triggers and application invariants

⚠ This is the weakest and most misunderstood letter. The DB only enforces the constraints **you declared**. "Debits equal credits" is only guaranteed if you expressed it as a constraint or wrote correct code. Consistency is largely the *application's* responsibility; A, I and D are the engine's.

I — Isolation

Concurrent transactions do not observe each other's partial work. Full isolation = **serializability**: the result equals *some* serial order of the transactions. In practice you dial this down for throughput → §8.

Implemented by locking (§7) and/or MVCC.

D — Durability

Once COMMIT returns, the change survives a crash.

How InnoDB does it: the **redo log** (`ib_logfile*`) + **WAL** (write-ahead logging). Changes are written to the redo log and `fsync`ed *before* commit returns; the dirty data pages are flushed to the tablespace lazily afterwards. On crash recovery, redo is replayed forward, then undo rolls back uncommitted work.

Key knob:

```
innodb_flush_log_at_trx_commit
  1  = flush+fsync at every commit      → fully durable (default, ACID)
  2  = write to OS cache each commit, fsync once/sec  → survives mysqld crash, not OS crash
  0  = write+fsync once per second      → fastest, can lose ~1s of commits
```

Plus `sync_binlog=1` for binlog durability (matters for replication correctness).

PostgreSQL equivalents: `fsync`, `synchronous_commit`, `wal_level`.

3.2 ACID vs BASE

ACID (relational)	BASE (many NoSQL)
Atomicity	**B**asically **A**vailable
Consistency	**S**oft state
Isolation	**E**ventual consistency
Durability	

BASE trades immediate consistency for availability and partition tolerance (§11). Not "worse" — different problem.

3.3 Recall card

> **A**tomicity → undo log. **C**onsistency → your constraints, app's job. **I**solation → MVCC + locks. **D**urability → redo log / WAL + fsync. A, I, D are engine guarantees; C is a contract you write.

3.4 Practice

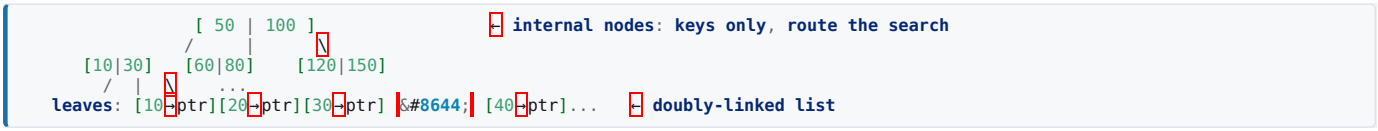
1. Which log gives atomicity and which gives durability? What happens to each at crash recovery?
2. `innodb_flush_log_at_trx_commit=2` — exactly what can you lose, and in which failure mode?
3. Why is Consistency the odd one out in ACID?

4. Indexes

4.1 Mental model

An index is a **sorted, separately-stored copy of some columns plus a pointer to the row**. It converts an O(n) table scan into an O(log n) tree descent. It costs write throughput and disk.

4.2 B+Tree — the default



Properties that explain *everything* about index behaviour:

- All data lives in **leaves**; internal nodes only route → high fan-out → typically **3–4 levels** for hundreds of millions of rows.
- Leaves are **linked in sorted order** → range scans (`BETWEEN`, `>`, `ORDER BY`, `LIMIT`) are sequential and cheap.
- Because it's sorted, an index can satisfy `ORDER BY` and `GROUP BY` **without a sort step**.

4.3 Clustered vs secondary (InnoDB)

- **Clustered index** = the table itself, physically ordered by the PK. Leaf node *contains the full row*. One per table.
- No PK declared? InnoDB uses the first `NOT NULL UNIQUE` index, else a hidden 6-byte rowid.
- **Secondary index** leaf stores `(indexed columns, primary key value)` — **not** a physical pointer.
- Consequence: a secondary index lookup that needs extra columns does a **second** descent into the clustered index. This is the **bookmark lookup / table access by index rowid**.
- Consequence: a fat PK (e.g. UUID char(36)) bloats *every* secondary index.

PK choice

- Prefer small, monotonically increasing (`BIGINT AUTO_INCREMENT`) → appends to the right-most leaf, minimal page splits.
- Random UUIDv4 PK → inserts scatter across the tree → page splits, fragmentation, poor cache locality. If you need UUIDs, use **UUIDv7 / ULID** (time-ordered) or keep an internal auto-inc PK and a unique UUID column for external IDs.
- Your hashid-style public IDs fit that second pattern perfectly: `id BIGINT` internal PK + `public_id VARCHAR UNIQUE`.

PostgreSQL note: heap-organised, not clustered. All indexes point to a heap tuple `(page, offset)`; `CLUSTER` is a one-off physical reorder, not maintained.

4.4 Composite indexes and the leftmost-prefix rule

`INDEX idx (trip_id, status, created_at)` can serve:

Query predicate	Uses index?
`WHERE trip_id=?`	✓
`WHERE trip_id=? AND status=?`	✓
`WHERE trip_id=? AND status=? AND created_at>?`	✓ full
`WHERE trip_id=? AND created_at>?`	⚠ partial — only `trip_id` narrows; `created_at` filtered after
`WHERE status=?`	✗ no leftmost prefix
`WHERE trip_id=? ORDER BY created_at`	⚠ needs `status` to be an equality to avoid filesort

Ordering rule of thumb: equality columns first, then the range/sort column last. A range predicate stops the index from being used for anything to its right.

4.5 Covering index

If the index contains **every column the query touches**, the engine never visits the table.

```sql
SELECT status, created_at FROM ticket WHERE trip_id = 91;
CREATE INDEX idx_cover ON ticket (trip_id, status, created_at);
-- EXPLAIN → Extra: "Using index"   ← covering, no bookmark lookup
```

PostgreSQL: `CREATE INDEX ... INCLUDE (col)` adds payload columns to leaves without making them part of the key.

4.6 Selectivity & cardinality

- **Cardinality** = number of distinct values. **Selectivity** = cardinality / row count (0→1).
- High selectivity (`email`, `ticket_no`) → great index.
- Low selectivity (`gender`, `is_active`, `status` with 3 values) → the optimizer will usually prefer a full scan, because random I/O for >~20–30% of rows beats sequential.

- Fix for low-selectivity columns: put them *after* a selective column in a composite index, or use a **partial index** (PG: `WHERE status='pending'`) which indexes only the rare rows.

4.7 Index types beyond B+Tree

Type	Engine	Good for	Not for
B+Tree	all	equality, range, sort, prefix `LIKE 'abc%'`	leading wildcard
Hash	MEMORY, PG `USING HASH`, InnoDB adaptive (internal)	`=` only	ranges, sorting
Full-text	MySQL FULLTEXT, PG `tsvector` + GIN	natural-language search	structured filters
GIN	PostgreSQL	JSONB, arrays, full-text (many keys per row)	high write churn
GiST / SP-GiST	PostgreSQL	geometry, ranges, nearest-neighbour	
BRIN	PostgreSQL	huge naturally-ordered tables (time-series); tiny index	unordered data
Bitmap	Oracle/warehouses	low-cardinality analytics	OLTP writes
Spatial (R-Tree)	MySQL `SPATIAL`	geo bounding boxes	

Other flavours: **unique** (constraint + index), **partial/filtered**, **functional/expression** (`CREATE INDEX ON users (LOWER(email))`), **descending**, **prefix index** (`INDEX (url(100))` — MySQL only; can't be covering).

4.8 When an index is silently NOT used

1. **Function on the column** — `WHERE YEAR(created_at)=2026` ✗ → `WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01'` ✓ (this is *sargability*).
2. **Leading wildcard** — `LIKE '%abc'` ✗ (need full-text or a reversed-string index).
3. **Type mismatch / implicit cast** — `WHERE phone = 8801711...` on a `VARCHAR` column casts the *column*, killing the index. Quote it.
4. **Collation mismatch** on a join column.
5. `OR` across different columns → often two scans; rewrite as `UNION ALL` or add an index-merge-friendly design.
6. **Low selectivity** — optimizer chooses a scan on purpose.
7. `!=`, `NOT IN`, `IS NOT NULL` on most rows.
8. **Stale statistics** — `ANALYZE TABLE t;`
9. Small table — a scan is genuinely faster.

4.9 The cost of indexes

- Every `INSERT` / `UPDATE` / `DELETE` must maintain **every** affected index → write amplification.
- Disk + buffer-pool memory pressure (indexes compete with data for the pool).
- Redundant indexes: `(a)` is useless if `(a,b)` exists. Audit with: `sql SELECT * FROM sys.schema_unused_indexes; SELECT * FROM sys.schema_redundant_indexes;`
- Rule: index for your **actual query patterns**, verified by the slow query log — not speculatively.

4.10 Recall card

> B+Tree: sorted leaves, linked, 3–4 levels. InnoDB clustered PK holds the row; secondary index holds PK → bookmark lookup. Leftmost prefix; equality columns first, range last. Covering index = "Using index". Sargable predicates only — no functions on indexed columns. Every index taxes writes.

4.11 Practice

1. `INDEX (a,b,c)`. Which of these use it fully: `WHERE a=1 AND b=2`, `WHERE b=2`, `WHERE a=1 AND c=3`, `WHERE a=1 AND b>2 AND c=3`?
2. Why does a UUIDv4 primary key hurt InnoDB more than PostgreSQL?
3. Rewrite `WHERE DATE(paid_at) = CURDATE()` to be sargable.
4. You have `INDEX(user_id)` and `INDEX(user_id, status)`. Which do you drop and why?

5. Query Optimization

5.1 How the optimizer works

```
SQL text → Parser → Rewriter (view expansion, subquery flattening, predicate pushdown)
        → Optimizer (cost-based: pick access path, join order, join algorithm)
        → Execution plan → Executor
```

The optimizer is **cost-based**: it estimates rows and I/O from **statistics** (histograms, index cardinality) and picks the cheapest plan. Bad statistics → bad plan. `ANALYZE TABLE t;` / PG `ANALYZE;`.

5.2 Reading EXPLAIN (MySQL)

```
EXPLAIN SELECT ... ;
EXPLAIN ANALYZE SELECT ... ;   -- MySQL 8.0.18+ / PG: actually runs it, gives real timings
EXPLAIN FORMAT=JSON SELECT ...; -- full cost detail
```

`type` column — access method, best → worst:

type	Meaning
`system` / `const`	≤1 row, PK/unique lookup with constant
`eq_ref`	one row per row from previous table (PK/unique join) — ideal for joins
`ref`	non-unique index lookup, several rows
`range`	index range scan (`BETWEEN` , `>` , `IN`)
`index`	full **index** scan (all index entries) — better than ALL, still bad
`ALL`	**full table scan** — the red flag

`Extra` column — what to look for:

Extra	Meaning
`Using index`	✓ covering index, no table access
`Using where`	filtering after fetching rows — fine, but check `rows`
`Using index condition`	✓ Index Condition Pushdown — filter applied in the index layer
`Using filesort`	⚠ sort not satisfied by an index (may be in-memory or on disk)
`Using temporary`	⚠ temp table built — common with `GROUP BY` + `ORDER BY` on different columns
`Using join buffer (hash join)`	join without a usable index

Also watch: `rows` (estimated rows examined) and `filtered` (% surviving `WHERE`). `rows × filtered` should approximate the real output. A huge `rows` for a tiny result = missing index.

PostgreSQL equivalents: `Seq Scan` (=ALL), `Index Scan`, `Index Only Scan` (=covering), `Bitmap Heap Scan`, `Nested Loop` / `Hash Join` / `Merge Join`, and `Sort` / `HashAggregate`. Read `EXPLAIN (ANALYZE, BUFFERS)` bottom-up and compare **estimated vs actual rows** — a large mismatch means stale/insufficient stats.

5.3 Rewrite patterns that actually matter

1. Sargability — never wrap the indexed column

```
-- &#10007;           WHERE YEAR(created_at) = 2026
-- &#10003;           WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01'
-- &#10007;           WHERE CONCAT(first,' ',last) = 'A B'
-- &#10003;           functional index, or store a generated column
```

2. Keyset (seek) pagination instead of OFFSET

```
-- &#10007; page 5000: the DB reads and discards 100 000 rows
SELECT * FROM ticket ORDER BY id DESC LIMIT 20 OFFSET 100000;
-- &#10003; O(log n) regardless of depth
SELECT * FROM ticket WHERE id < :last_seen_id ORDER BY id DESC LIMIT 20;
```

Laravel: `Model::cursorPaginate(20)` does exactly this.

3. Deferred join for unavoidable deep OFFSET

```
SELECT t.* FROM ticket t
JOIN (SELECT id FROM ticket ORDER BY id LIMIT 20 OFFSET 100000) x USING (id);
```

The inner query is covered by the PK index; only 20 full rows are fetched.

4. `SELECT *` **→ explicit columns** Blocks covering indexes, drags TEXT/BLOB over the wire, breaks when schema changes.

5. `EXISTS` **vs** `IN` **vs** `JOIN`

- `IN (subquery)` with NULLs in the subquery → `NOT IN` returns nothing. Use `NOT EXISTS`.

- `EXISTS` short-circuits on the first match; usually best for semi-joins on large inner sets.

6. `UNION ALL` **over** `UNION` unless you truly need dedupe (`UNION` sorts).

7. `COUNT(*)` **on big tables** is O(n) in InnoDB and PostgreSQL (MVCC — no stored row count). Options: approximate from `information_schema` / `pg_class.reltuples`, maintain a counter, or `LIMIT`-based "has more" instead of a total. Laravel: `simplePaginate()` skips the count query.

8. Batch instead of loop — one `INSERT ... VALUES (...),(...),(...)` or `upsert()` beats 1000 round-trips.

9. Index hints as a last resort — `USE INDEX` / `FORCE INDEX`. Fix the stats/query first; hints rot.

5.4 The N+1 problem (biggest real-world win)

```
// &#10007; 1 + N queries
$trips = Trip::all();
foreach ($trips as $t) { echo $t->route->name; }

// &#10003; 2 queries
$trips = Trip::with('route')->get();

// &#10003; constrained + column-limited eager load
Trip::with(['tickets' => fn($q) => $q->select('id','trip_id','price')->where('status','sold')])->get();

// &#10003; aggregate without loading children
Trip::withCount('tickets')->get();

// &#10003; fail loudly in dev
Model::preventLazyLoading(! app()->isProduction());
```

5.5 Tooling / where to look

- MySQL slow query log (`long_query_time = 0.5`) + `pt-query-digest`
- `performance_schema.events_statements_summary_by_digest` — top queries by total time
- `sys.schema_tables_with_full_table_scans`, `sys.statements_with_temp_tables`
- PostgreSQL: `pg_stat_statements`, `auto_explain`
- Laravel: Telescope, Debugbar, `DB::listen()`
- **Optimize by total time (calls × avg), not worst single query.**

5.6 Recall card

> Cost-based optimizer driven by statistics. EXPLAIN: `type` ALL = red flag, want `const/eq_ref/ref/range`; `Extra` wants `Using index`, fears `filesort` / `temporary`. Keep predicates sargable, use keyset pagination, kill N+1, avoid `SELECT *`, batch writes. Profile with the slow log; optimize by total time.

5.7 Practice

1. Interpret: `type=ALL, rows=2,400,000, Extra=Using where; Using filesort`. What are your two fixes?
2. Why is `LIMIT 20 OFFSET 200000` slow, and give two fixes.
3. When does `NOT IN` silently return zero rows?
4. Query is fast in dev, slow in prod with identical schema. Name three causes.

6. Joins

6.1 Logical join types

Given `A` (left) and `B` (right):

Join	Returns
`INNER JOIN`	rows matching in both
`LEFT [OUTER] JOIN`	all A + matched B, NULLs where no match
`RIGHT JOIN`	mirror of LEFT (just flip the tables; rarely used)
`FULL OUTER JOIN`	all rows from both, NULL-padded. **Not in MySQL** — emulate with `LEFT UNION RIGHT`
`CROSS JOIN`	cartesian product, A × B
`SELF JOIN`	table joined to itself (hierarchies, adjacency: `employee.manager_id`)
`NATURAL JOIN`	auto-joins on same-named columns — **avoid**, breaks silently when a column is added
`USING (col)`	shorthand when the column names match, one output column

Derived / non-standard-name joins

- **Semi-join** — "A rows that have a match in B" → `WHERE EXISTS (...)` / `IN`. No B columns, no row multiplication.
- **Anti-join** — "A rows with no match in B" → `NOT EXISTS` / `LEFT JOIN ... WHERE b.id IS NULL`.
- **Lateral join** (`LATERAL` / MySQL 8 `LATERAL`, PG `CROSS/LEFT JOIN LATERAL`) — right side can reference the left side. Perfect for **top-N-per-group**.

```sql
-- Latest 3 tickets per trip
SELECT t.id, x.*
FROM trip t
JOIN LATERAL (
  SELECT * FROM ticket WHERE trip_id = t.id ORDER BY created_at DESC LIMIT 3
) x ON TRUE;
```

6.2 Physical join algorithms

Algorithm	How	Best when	Cost
Nested Loop Join	for each outer row, probe inner	inner side has an index on the join key; small outer	O(N × log M) with index, O(N×M) without
Block Nested Loop	buffer outer rows, scan inner once per block	no index, older MySQL	O(N×M / block)
Hash Join	build hash table on smaller side, probe with larger	large, unindexed, **equi-join only**	O(N+M), needs memory
Sort-Merge Join	sort both, merge in one pass	both already sorted / huge inputs, supports `<`, `>`	O(N log N + M log M)
Index Merge	intersect/union results of several indexes	multiple `OR` / `AND` predicates	

MySQL 8.0.18+ has real hash join and dropped BNL for it in 8.0.20. PostgreSQL has had all three for a long time and picks by cost.

6.3 Rules that bite

1. `ON` vs `WHERE` on an outer join — not interchangeable

```sql
-- Keeps all trips, filters which tickets attach
SELECT * FROM trip t LEFT JOIN ticket k ON k.trip_id=t.id AND k.status='sold';
-- Silently becomes an INNER JOIN: NULL <> 'sold'
SELECT * FROM trip t LEFT JOIN ticket k ON k.trip_id=t.id WHERE k.status='sold';
```

2. Row multiplication breaks aggregates Joining two one-to-many tables to the same parent multiplies rows → `SUM()` is inflated. Fix with separate subquery aggregates, or `SUM(DISTINCT ...)` only if values are unique, or a `LATERAL`.

3. Join columns must have identical type + collation + an index, or you get a full scan per outer row.

4. Join order — the optimizer reorders freely for inner joins; outer joins constrain the order. `STRAIGHT_JOIN` forces MySQL's order (last resort).

5. Driving table — the optimizer wants the **smallest filtered result set first**, then probes indexed inner tables.

6.4 Recall card

> Logical: INNER / LEFT / RIGHT / FULL / CROSS / SELF; semi (`EXISTS`), anti (`NOT EXISTS`), LATERAL for top-N-per-group. Physical: nested loop (indexed inner), hash (big equi-join), merge (sorted/huge). On an outer join, a filter on the right table belongs in `ON`, not `WHERE`.

6.5 Practice

1. Rewrite "trips that sold zero tickets" three ways (`NOT EXISTS`, `LEFT JOIN ... IS NULL`, `NOT IN`) and say which you'd ship.
2. Why does `LEFT JOIN ... WHERE right.col = 'x'` become an inner join?
3. You join `trip → tickets` and `trip → expenses`, then `SUM(expenses.amount)`. Why is the total wrong, and how do you fix it?
4. Which join algorithm can't do `a.x < b.y`?

7. Locks

7.1 Lock modes

Mode	Symbol	Compatible with
Shared (S) — read lock	`SELECT ... FOR SHARE` / `LOCK IN SHARE MODE`	other S
Exclusive (X) — write lock	`SELECT ... FOR UPDATE`, `UPDATE`, `DELETE`	nothing
Intention Shared (IS) / **Intention Exclusive (IX)**	table-level flags announcing "I hold row locks below"	IS/IX with each other

Compatibility matrix:

```
      IS    IX    S     X
IS    &#10003;     &#10003;     &#10003;     &#10007;
IX    &#10003;     &#10003;     &#10007;     &#10007;
S     &#10003;     &#10007;     &#10003;     &#10007;
X     &#10007;     &#10007;     &#10007;     &#10007;
```

Intention locks exist so a table-level lock request can check conflicts in O(1) instead of scanning every row lock.

7.2 Granularity

Table → Page → **Row** (InnoDB) . Finer granularity = more concurrency, more lock bookkeeping overhead.

- MyISAM = table-level only (why it's dead for OLTP).
- InnoDB = row-level, but **locks index records, not rows**. Huge consequence → next section.

7.3 InnoDB row lock types (the interview favourite)

- **Record lock** — locks a single index record.
- **Gap lock** — locks the *interval between* index records, so no one can insert there. Prevents phantoms.
- **Next-key lock** — record lock + gap before it. **This is InnoDB's default at REPEATABLE READ.**
- **Insert intention lock** — a gap lock that signals intent to insert; multiple inserts into the same gap don't block each other unless they collide on the same value.

⚠ **Because locks are on index records, a query with no usable index locks effectively every row it scans.** `UPDATE ticket SET status='x' WHERE some_unindexed_col=5` on a million-row table will lock the table in practice. Always make sure your `WHERE` on a write is indexed.

Gap locks disappear at READ COMMITTED (only record locks remain) — which is why RC has higher concurrency and fewer deadlocks, at the cost of phantoms.

7.4 Pessimistic vs optimistic

Pessimistic — take the lock up front. Correct under contention, serializes throughput.

```
$seat = TripSeat::where('trip_id',$id)->where('seat_no','A1')
        ->lockForUpdate()->first();     // SELECT ... FOR UPDATE  (X lock)
// ->sharedLock()                        // SELECT ... FOR SHARE   (S lock)
```

Modifiers: `FOR UPDATE NOWAIT` (fail instantly), `FOR UPDATE SKIP LOCKED` (skip contended rows — perfect for queue/job-claiming tables).

Optimistic — no lock; detect conflict at write time via a version column.

```
UPDATE trip_seat SET status='sold', version = version + 1
WHERE id = 42 AND version = 7;
-- affected rows = 0  →  someone else won; retry or 409
```

Best when conflicts are rare. Under real contention (last seat on a popular trip), pessimistic wins.

Also: a **UNIQUE constraint** is the cheapest correctness mechanism of all — `UNIQUE(trip_id, seat_no)` on the ticket table makes double-booking structurally impossible, no matter what the app does.

7.5 Deadlocks

Four Coffman conditions: mutual exclusion, hold-and-wait, no preemption, circular wait.

```
T1: lock seat A → wants seat B
T2: lock seat B → wants seat A       ← cycle
```

InnoDB detects the cycle and **rolls back the transaction with the least work done**, returning error 1213. It's normal and expected under load — your job is to **retry**.

Inspect: `SHOW ENGINE INNODB STATUS\G` → LATEST DETECTED DEADLOCK. Also `innodb_lock_wait_timeout` (default 50s) → error 1205 for simple waits (not the same as a deadlock).

Prevention

1. Always acquire locks in a **consistent order** (e.g. `ORDER BY id` before locking a set).
2. Keep transactions short; do I/O outside them.
3. Index the `WHERE` of every write so you lock fewer records.
4. Lower isolation to READ COMMITTED if you don't need gap locks.
5. Retry on 1213 — Laravel's `DB::transaction($cb, $attempts)` already does.

7.6 Two-Phase Locking (2PL)

Theoretical basis of serializability.

- **Growing phase**: acquire locks, never release.
- **Shrinking phase**: release locks, never acquire.
- **Strict 2PL**: hold all X locks until commit/abort → guarantees serializability **and** recoverability (no cascading aborts). This is what real engines implement.

7.7 Diagnostics

```
SELECT * FROM performance_schema.data_locks;          -- what is locked
SELECT * FROM performance_schema.data_lock_waits;     -- who waits on whom
SELECT * FROM sys.innodb_lock_waits;                  -- readable summary
SHOW ENGINE INNODB STATUS\G
```

7.8 Recall card

> S/X + intention locks (IS/IX). InnoDB locks **index records**: record, gap, next-key (default at RR), insert intention. No index on a write ⇒ lock everything scanned. Pessimistic (`FOR UPDATE`) vs optimistic (version column) vs a UNIQUE constraint. Deadlock = cycle → engine kills the cheaper txn → retry. Fix by consistent lock order + short transactions. Strict 2PL underpins serializability.

7.9 Practice

1. Why does `UPDATE t SET x=1 WHERE unindexed_col=5` block the whole table?
2. What does `SKIP LOCKED` solve? Sketch a job-claiming query.
3. Two workers deadlock updating the same three rows in different orders. Two fixes.
4. Design seat booking three ways (pessimistic / optimistic / unique constraint) and pick one.

8. Isolation Levels

8.1 The read phenomena

Anomaly	What happens
Dirty read	You read data another transaction wrote but hasn't committed (it may roll back)
Non-repeatable read	You read the *same row* twice in one transaction and get different values, because another txn committed an UPDATE between
Phantom read	You run the *same range query* twice and get a different *set of rows*, because another txn committed an INSERT/DELETE
Lost update	Two read-modify-write cycles overlap; the second overwrites the first (`balance = balance - 100` done in app code)
Write skew	Two txns each read a set, each makes a decision that's valid alone, but together violate an invariant. Not preventable below SERIALIZABLE
Read skew	You read A and B at different points and see an inconsistent pair

8.2 The four SQL standard levels

Level	Dirty	Non-repeatable	Phantom
READ UNCOMMITTED	✗ possible	✗	✗
READ COMMITTED	✓ prevented	✗	✗
REPEATABLE READ	✓	✓	✗ (standard) / ✓ in InnoDB
SERIALIZABLE	✓	✓	✓

Defaults: MySQL/InnoDB → **REPEATABLE READ**. PostgreSQL, Oracle, SQL Server → **READ COMMITTED**.

```
SELECT @@transaction_isolation;
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;   -- next txn only
```

8.3 MVCC — how reads avoid locks

Multi-Version Concurrency Control: instead of locking rows for reads, keep multiple versions and show each transaction a consistent snapshot. **Readers never block writers; writers never block readers.**

InnoDB implementation:

- Every row carries hidden `DB_TRX_ID` (last writer) and `DB_ROLL_PTR` (pointer to the previous version in the undo log).
- A transaction builds a **read view** = the set of transaction IDs active at snapshot time.
- Reading a row: if its `DB_TRX_ID` isn't visible to your read view, follow `DB_ROLL_PTR` back through the undo log until you find a version that is.
- **REPEATABLE READ**: snapshot taken at the *first* read, reused for the whole transaction.
- **READ COMMITTED**: a *new* snapshot for every statement — hence non-repeatable reads.

Consequences worth knowing:

- A long-running transaction pins old undo versions → **undo log / history list bloat** (PG equivalent: dead tuples, vacuum can't reclaim). This is the real reason "keep transactions short" matters at scale.
- **Consistent (snapshot) read** = plain `SELECT`, uses MVCC, no locks.
- **Locking read** = `SELECT ... FOR UPDATE / FOR SHARE`, `UPDATE`, `DELETE` — these read the **latest committed version**, not your snapshot. This is why you can get a "surprising" value from `UPDATE ... SET x = x + 1` inside RR.

8.4 InnoDB's REPEATABLE READ specifics

- Prevents phantoms for plain reads via MVCC snapshot, **and** for locking reads via **next-key (gap) locks**. So InnoDB RR is stronger than the SQL standard requires.
- Still allows **lost updates** if you do read-modify-write in application code. Prevent with `FOR UPDATE`, an atomic `SET col = col - ?`, or a version column.
- Still allows **write skew** — needs SERIALIZABLE.

8.5 SERIALIZABLE, two flavours

- **MySQL/InnoDB**: implicitly converts every plain `SELECT` to `SELECT ... FOR SHARE`. Pessimistic, lots of blocking.
- **PostgreSQL**: **SSI** (Serializable Snapshot Isolation) — optimistic. Runs at snapshot isolation, tracks read/write dependency cycles, and aborts one transaction with `40001 serialization_failure` if a cycle would break serializability. You **must** be prepared to retry.

8.6 Choosing

Need	Level
Analytics/report tolerance for slight staleness	READ COMMITTED
Typical web CRUD	RC or RR (default is fine)
A transaction that reads the same rows repeatedly and must see a stable picture	REPEATABLE READ
Financial invariants across rows ("total balance ≥ 0", "at most one on-call doctor")	SERIALIZABLE + retry, or explicit locking
Never	READ UNCOMMITTED

Practical note: switching MySQL from RR to RC reduces gap locking and deadlocks and is common in high-throughput shops — but check that your app doesn't rely on stable snapshots, and note RC requires `binlog_format=ROW`.

8.7 Recall card

> Anomalies: dirty → non-repeatable → phantom (+ lost update, write skew). RU / RC / RR / SERIALIZABLE. MySQL defaults RR (and blocks phantoms via next-key locks); PG defaults RC. MVCC = versions + read view + undo log → readers don't block writers. RR snapshot = first read; RC snapshot = per statement. Locking reads always see the latest committed row. Lost update needs `FOR UPDATE`/atomic UPDATE/version; write skew needs SERIALIZABLE.

8.8 Practice

1. Give a concrete write-skew scenario and explain why RR doesn't stop it.
2. Under RR, T1 reads balance=100, T2 commits balance=50, T1 runs `UPDATE ... SET balance = balance - 10`. What's stored? Why?
3. Why is a 40-minute open transaction bad even if it only reads?
4. What exactly does InnoDB SERIALIZABLE do to your plain SELECTs?

9. Sharding

9.1 Scaling ladder (do these in order)

1. Indexes + query fixes (free, biggest win)
2. Caching (Redis, query cache, HTTP cache)
3. Vertical scaling (bigger box)
4. **Read replicas** (§10) — scales reads only
5. **Partitioning** (single server, multiple physical segments)
6. **Sharding** (multiple servers/databases) — last resort, permanent complexity

9.2 Partitioning vs sharding

	Partitioning	Sharding
Scope	one server, one logical table split into segments	many servers, each holding a subset
Transparency	DB handles it; app sees one table	app or middleware routes queries
Solves	manageability, partition pruning, cheap bulk delete of old data	write throughput, storage, memory

Vertical partitioning — split *columns* (move a rarely-read `TEXT` blob to a side table). **Horizontal partitioning** — split *rows*. Sharding is horizontal partitioning across machines.

MySQL native partitioning: `PARTITION BY RANGE (YEAR(created_at))`, `HASH`, `KEY`, `LIST`. Great for time-series retention (`ALTER TABLE ... DROP PARTITION p2023` is instant vs a huge `DELETE`).

9.3 Shard key — the single most important decision

The shard key determines which shard a row lives on. **Changing it later means a full migration**, so choose deliberately.

Criteria:

- **High cardinality** — enough distinct values to spread across shards.
- **Even distribution** — no hotspots.
- **Present in most queries** — otherwise every query is a scatter-gather across all shards.
- **Stable** — a key that changes forces a row to move shards.

For a ticketing system, `operator_id` or `route_region` is a natural shard key if operators are independent; `user_id` is natural if user-centric queries dominate. You usually can't have both.

9.4 Sharding strategies

a) Range-based — `user_id 1–1M → shard1`, `1M–2M → shard2`

- ✓ simple, range queries stay local, easy to add a shard at the end
- ✗ hotspots (newest range gets all writes), uneven growth

b) Hash-based — `shard = hash(user_id) % N`

- ✓ even distribution
- ✗ range queries scatter; **changing N rehashes almost everything**

c) Consistent hashing — map shards and keys onto a ring; a key belongs to the next shard clockwise. Virtual nodes even out the load.

- ✓ adding/removing a shard moves only ~1/N of keys
- Used by Cassandra, DynamoDB, Riak, memcached clients

d) Directory / lookup-based — a mapping service says which shard holds a key

- ✓ maximum flexibility, easy rebalancing, supports tenant pinning
- ✗ the directory is a lookup on every query and a single point of failure (must be cached + HA)

e) Geo / entity-group based — shard by region or by tenant. Common in multi-tenant SaaS: one schema/DB per tenant. Simple mental model, but thousands of tenants = thousands of schemas to migrate.

9.5 What sharding costs you

- **Cross-shard joins** — gone. You do application-side joins, or denormalize, or keep small reference tables replicated to every shard.
- **Cross-shard transactions** — no single-node ACID. You need 2PC (slow, blocking on coordinator failure) or the **Saga pattern** (a sequence of local transactions + compensating actions) with an outbox for reliable events.
- **Global uniqueness / IDs** — `AUTO_INCREMENT` no longer works. Use UUIDv7/ULID, Snowflake IDs (timestamp + machine id + sequence), or per-shard offsets (`auto_increment_increment` / `offset`).
- **Global secondary indexes** — a query on a non-shard-key column hits every shard (scatter-gather). Latency = the slowest shard.

- **Aggregates / reporting** — must fan out and merge; usually solved by shipping everything to a separate OLAP store.
- **Rebalancing & resharding** — the hardest operational problem. Mitigate by starting with many **logical shards** (e.g. 1024) mapped onto few physical servers, then moving logical shards as you grow.
- **Schema migrations** — N times the work, plus mixed-version windows.
- **Referential integrity** — cross-shard FKs don't exist.

9.6 Hotspots and the celebrity problem

One tenant/route/celebrity generates 40% of traffic. Mitigations: dedicated shard for that key, composite shard key (`operator_id + date_bucket`), salting the key, or caching in front.

9.7 Recall card

> Order: index → cache → vertical → replicas → partition → shard. Vertical = columns, horizontal = rows. Strategies: range (hotspots), hash (rehash pain), consistent hashing (~1/N moves), directory (flexible, extra hop), geo/tenant. Shard key: high cardinality, even, in most queries, stable. Costs: no cross-shard joins/transactions/FKs, scatter-gather, ID generation, resharding. Use many logical shards from day one.

9.8 Practice

1. Why does `hash(id) % N` make adding a shard painful, and how does consistent hashing fix it?
2. Pick a shard key for a multi-tenant ticketing platform and justify it against all four criteria.
3. Compare 2PC vs Saga for a cross-shard "book seat + charge wallet" operation.
4. What is scatter-gather and when does it dominate your p99?

10. Replication

10.1 Purpose

- **Read scale** — fan reads out to replicas
- **High availability** — promote a replica on primary failure
- **Durability / DR** — geographically separate copy
- **Offload** — run analytics/backups on a replica without touching production

Replication does **not** scale writes (every replica applies every write). That's sharding's job.

10.2 Topologies

Topology	Notes
Primary → Replica(s)	the standard. One writer, N readers
Chained / cascading	replica of a replica; reduces load on the primary, adds lag
Multi-primary (master-master)	both accept writes → **conflict resolution required**. Use only with care (e.g. ring with distinct auto-inc offsets)
Group Replication / Galera	synchronous multi-primary with certification-based conflict detection and automatic membership
Quorum (Raft/Paxos)	write succeeds when a majority acks — CockroachDB, etcd, MongoDB replica sets, Spanner

10.3 Synchronicity

Mode	Primary waits for	Data loss on failover	Latency
Asynchronous	nothing (default in MySQL)	possible — unshipped writes lost	lowest
Semi-synchronous	≥1 replica to **receive** (not apply)	much reduced	+1 RTT
Synchronous	replica(s) to commit	none	highest, and unavailable if a replica is down

MySQL: `rpl_semi_sync_source_enabled`. PostgreSQL: `synchronous_commit = on|remote_write|remote_apply` + `synchronous_standby_names`.

10.4 How MySQL does it

1. Primary writes changes to the **binary log** (binlog).
2. Replica's **I/O thread** pulls the binlog → writes to its **relay log**.
3. Replica's **SQL (applier) thread** replays the relay log.

Binlog formats

- `STATEMENT` — logs the SQL. Compact, but non-deterministic functions (`NOW()`, `UUID()`, `RAND()`, `LIMIT` without ORDER BY) diverge.
- `ROW` — logs the actual before/after row images. **Safe, the default and the right choice.** Bigger logs.
- `MIXED` — statement, switching to row when unsafe.

GTIDs (Global Transaction IDs) give each transaction a unique ID, making failover and re-pointing replicas vastly easier than tracking `(file, position)`.

PostgreSQL equivalents: **streaming replication** ships the WAL; **logical replication** (publications/subscriptions) ships row changes and allows selective/cross-version replication.

10.5 Replication lag — the thing that will actually bite you

Causes: single-threaded appliers (fixed by parallel replication / `replica_parallel_workers`), long transactions, big `ALTER TABLE`, network, replica hardware, replica running heavy analytics.

Measure: `SHOW REPLICA STATUS` → `Seconds_Behind_Source` (crude), or a heartbeat table (accurate). PG: `pg_last_xact_replay_timestamp()`.

The read-after-write problem

```
POST /tickets  → writes to primary
GET  /tickets  → reads from a replica 300ms behind → "my ticket disappeared"
```

Fixes:

1. **Read your own writes**: route a user's reads to the primary for N seconds after they write. Laravel's `sticky => true` option does exactly this per request lifecycle.
2. **Monotonic reads**: pin a session to one replica.
3. Wait for a position: `SELECT WAIT_FOR_EXECUTED_GTID_SET(...)`.
4. Just read from the primary for anything write-adjacent.

Laravel config:

```
'mysql' => [
  'read'   => ['host' => ['10.0.0.2','10.0.0.3']],
  'write'  => ['host' => ['10.0.0.1']],
  'sticky' => true,
]
```

10.6 Failover

- **Manual vs automatic** (Orchestrator, MHA, MySQL InnoDB Cluster, Patroni for PG, RDS Multi-AZ).
- **Split-brain**: two nodes both think they're primary → divergent writes. Prevent with quorum/fencing (STONITH), never with a naive 2-node auto-failover.
- Always test: promote, re-point replicas, update the app's write endpoint (usually via a VIP/proxy — ProxySQL, HAProxy, RDS endpoint).

10.7 Recall card

> Replication scales reads + HA, never writes. MySQL: binlog → relay log → applier; use `ROW` format + GTIDs. Async (fast, can lose data) / semi-sync (waits for receipt) / sync (no loss, slower). Lag causes read-after-write bugs → sticky reads to the primary. Multi-primary needs conflict resolution; quorum systems need a majority. Guard against split-brain.

10.8 Practice

1. Why is `STATEMENT` binlog format unsafe? Give two statements that break.
2. A user books a ticket and the confirmation page shows nothing. Diagnose and give three fixes.
3. Semi-sync waits for the replica to *receive*, not *apply*. What can still be lost?
4. Why doesn't adding replicas help a write-bound workload?

11. CAP Theorem

11.1 Statement

In a distributed data store you can guarantee at most **two** of:

- **C — Consistency** (here: *linearizability*): every read returns the most recent committed write, as if there were a single copy. **Note this is not the C in ACID.**
- **A — Availability**: every request to a non-failing node gets a non-error response (not necessarily the freshest).
- **P — Partition tolerance**: the system keeps operating despite arbitrary message loss between nodes.

11.2 The correct framing

Network partitions are a **fact of nature**, not a design choice. Any real distributed system must tolerate them. So the theorem is really:

> **When a partition occurs, do you choose Consistency or Availability?**

- **CP** — refuse writes/reads on the minority side to avoid divergence. You return errors, but never wrong data.
- **AP** — keep serving on both sides, accept divergence, reconcile later (last-write-wins, vector clocks, CRDTs, application merge).
- **CA** — only meaningful for a single node or a system that assumes no partitions. A single-node PostgreSQL is "CA" in a trivial sense; there's no such thing as a partition-intolerant *distributed* system that stays correct.

11.3 Where systems land

System	Lean	Note
PostgreSQL / MySQL (single primary)	CP-ish	primary down = unavailable writes
MySQL async replication	AP-ish in practice	replicas serve stale reads
MongoDB (replica set)	CP	majority write concern; minority can't elect
Cassandra / DynamoDB / Riak	AP (tunable)	quorum settings let you dial: `R + W > N` gives strong consistency
etcd / ZooKeeper / Consul	CP	Raft/ZAB, majority quorum
Google Spanner	"effectively CA"	CP with TrueTime + enough redundancy that partitions are vanishingly rare
Redis Sentinel / Cluster	AP-ish	can lose writes on failover

Tunable consistency (Dynamo-style): with N replicas, W write acks and R read acks, `R + W > N` guarantees a read overlaps a write → strong consistency. `N=3, W=2, R=2` is the classic setting.

11.4 PACELC — the more useful refinement

> **if P** (partition) → choose **A** or **C**; **E**lse (normal operation) → choose **L**atency or **C**onsistency.

This captures what CAP misses: even with no partition, replication forces a latency-vs-consistency trade-off every single day.

- Cassandra: **PA/EL** (available under partition, low latency normally)
- MongoDB: **PC/EC**
- DynamoDB: **PA/EL** (eventual reads) with an option for **EC** (strongly consistent reads, higher latency/cost)
- Spanner: **PC/EC** (pays latency for consistency via TrueTime)

11.5 Consistency models spectrum

```
Strong / Linearizable  →  Sequential  →  Causal  →  Read-your-writes /
Monotonic reads        →  Eventual
       (safer, slower)                        (faster, weaker)
```

"Eventual consistency" = if writes stop, all replicas converge. Says nothing about *when*.

11.6 Common misconceptions to correct in an interview

- ✗ "Pick 2 of 3." → You always have P; you pick C or A **during a partition**.
- ✗ "CAP's C is ACID's C." → No: CAP's C is linearizability; ACID's C is constraint validity.
- ✗ "NoSQL is AP, SQL is CP." → Both are configurable; MongoDB is CP, a MySQL cluster can be AP.
- ✗ "CAP applies at the system level." → It applies per operation; a single system can offer both strong and eventual reads.

11.7 Recall card

> C = linearizability, A = every node answers, P = survives message loss. P is mandatory → the real choice is **C or A during a partition**. CP refuses service; AP serves stale and reconciles. PACELC adds: else, Latency vs Consistency. `R + W > N` for quorum consistency. CAP's C ≠ ACID's C.

11.8 Practice

1. Why is "CA" not a real option for a distributed system?
2. `N=5`. Give W and R for strong consistency while maximising read speed.
3. Classify your own stack (MySQL primary + 2 replicas + Redis) in PACELC terms.
4. Give a business scenario where AP is clearly right, and one where CP is clearly right.

12. One-page cheat sheet

Normalization — 1NF atomic · 2NF no partial FD · 3NF no transitive FD · BCNF every determinant is a super key · 4NF no independent MVD. 3NF is the target; denormalize with a measured reason.

Transactions — atomic unit; Active→Committed/Aborted. MySQL autocommits and implicitly commits DDL; PG has transactional DDL. Savepoints = partial rollback = Laravel's nesting. No network I/O inside.

ACID — Atomicity=undo log · Consistency=your constraints · Isolation=MVCC+locks · Durability=redo/WAL+fsync (`innodb_flush_log_at_trx_commit=1`).

Indexes — B+Tree, sorted linked leaves, 3–4 levels. InnoDB clustered PK holds the row; secondary holds PK → bookmark lookup. Leftmost prefix; equality first, range last. Covering = "Using index". Keep predicates sargable. Indexes tax writes.

Query optimization — cost-based on statistics. EXPLAIN `type`: const/eq_ref/ref/range good, `ALL` bad; `Extra`: want `Using index`, fear `filesort`/`temporary`. Keyset pagination, kill N+1, batch writes, optimize by total time.

Joins — INNER/LEFT/RIGHT/FULL/CROSS/SELF + semi/anti/LATERAL. Algorithms: nested loop (indexed inner), hash (big equi-join), merge (sorted). Outer-join filters go in `ON`, not `WHERE`.

Locks — S/X + IS/IX. InnoDB locks index records: record/gap/next-key/insert-intention. Unindexed write ⇒ locks everything scanned. Pessimistic vs optimistic vs UNIQUE constraint. Deadlock → engine kills the cheaper txn → retry. Strict 2PL.

Isolation — dirty → non-repeatable → phantom (+ lost update, write skew). RU/RC/RR/SERIALIZABLE. MySQL default RR (blocks phantoms via next-key locks); PG default RC. MVCC = versions + read view + undo. Locking reads see the latest committed row.

Sharding — index→cache→vertical→replicas→partition→shard. Range / hash / consistent hashing / directory / geo. Shard key: high cardinality, even, in most queries, stable. Loses cross-shard joins, transactions, FKs, auto-inc.

Replication — scales reads and HA, never writes. binlog(ROW)+GTID → relay log → applier. async/semi-sync/sync. Lag ⇒ read-after-write bugs ⇒ sticky reads. Beware split-brain.

CAP — P is mandatory; choose C or A during a partition. PACELC: else Latency vs Consistency. `R+W>N`. CAP's C ≠ ACID's C.

13. Hands-on lab

Run these against a scratch MySQL 8 database. Everything below is safe to drop afterwards.

13.1 Setup

```
CREATE DATABASE dbms_lab; USE dbms_lab;

CREATE TABLE route (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  code        VARCHAR(20) NOT NULL UNIQUE,
  origin      VARCHAR(60) NOT NULL,
  destination VARCHAR(60) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE trip (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  route_id   BIGINT UNSIGNED NOT NULL,
  departs_at DATETIME NOT NULL,
  fare       DECIMAL(10,2) NOT NULL,
  CONSTRAINT fk_trip_route FOREIGN KEY (route_id) REFERENCES route(id)
) ENGINE=InnoDB;

CREATE TABLE trip_seat (
  id       BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  trip_id  BIGINT UNSIGNED NOT NULL,
  seat_no  VARCHAR(5) NOT NULL,
  status   ENUM('available','held','sold') NOT NULL DEFAULT 'available',
  version  INT UNSIGNED NOT NULL DEFAULT 0,
  UNIQUE KEY uq_trip_seat (trip_id, seat_no),
  CONSTRAINT fk_seat_trip FOREIGN KEY (trip_id) REFERENCES trip(id)
) ENGINE=InnoDB;

CREATE TABLE ticket (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  public_id   CHAR(26) NOT NULL UNIQUE,          -- ULID / hashid style external id
  trip_id     BIGINT UNSIGNED NOT NULL,
  seat_no     VARCHAR(5) NOT NULL,
  user_id     BIGINT UNSIGNED NOT NULL,
  price_paid  DECIMAL(10,2) NOT NULL,            -- historical fact, not denormalization
  status      ENUM('booked','cancelled') NOT NULL DEFAULT 'booked',
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_ticket_seat (trip_id, seat_no),  -- double-booking is structurally impossible
  CONSTRAINT fk_ticket_trip FOREIGN KEY (trip_id) REFERENCES trip(id)
) ENGINE=InnoDB;
```

Seed a few hundred thousand rows with a recursive CTE so EXPLAIN has something to chew on:

```
INSERT INTO route (code,origin,destination) VALUES
 ('DHK-CTG','Dhaka','Chattogram'),('DHK-SYL','Dhaka','Sylhet'),('DHK-KHL','Dhaka','Khulna');

INSERT INTO trip (route_id, departs_at, fare)
WITH RECURSIVE n(x) AS (SELECT 1 UNION ALL SELECT x+1 FROM n WHERE x < 5000)
SELECT 1 + (x % 3), NOW() + INTERVAL x HOUR, 800 + (x % 20) * 50 FROM n;

INSERT INTO ticket (public_id, trip_id, seat_no, user_id, price_paid, created_at)
WITH RECURSIVE n(x) AS (SELECT 1 UNION ALL SELECT x+1 FROM n WHERE x < 200000)
SELECT LPAD(x,26,'0'), 1 + (x % 5000), CONCAT(CHAR(65 + (x % 10)), 1 + (x % 40)),
       1 + (x % 9000), 800 + (x % 20) * 50, NOW() - INTERVAL x MINUTE
FROM n
ON DUPLICATE KEY UPDATE ticket.id = ticket.id;
```

13.2 Exercises

L1 — Index effect. Run `EXPLAIN SELECT * FROM ticket WHERE user_id = 4321;` Note `type` and `rows`. Add `CREATE INDEX idx_user ON ticket(user_id);` Re-run. Compare.

L2 — Covering index. `EXPLAIN SELECT trip_id, status FROM ticket WHERE user_id = 4321;` then `CREATE INDEX idx_cover ON ticket(user_id, trip_id, status);` — confirm `Extra: Using index`.

L3 — Leftmost prefix. With `INDEX(user_id, trip_id, status)`, EXPLAIN each of: `WHERE trip_id=7`, `WHERE user_id=1 AND status='booked'`, `WHERE user_id=1 AND trip_id=7 AND status='booked'`. Explain each result.

L4 — Sargability. Compare `WHERE DATE(created_at)='2026-08-10'` vs the range form. Look at `type` and `rows`.

L5 — Pagination. Time `LIMIT 20 OFFSET 150000` vs the keyset equivalent.

L6 — Locks. Two terminals:

```
-- session A
START TRANSACTION;
SELECT * FROM trip_seat WHERE trip_id=1 AND seat_no='A1' FOR UPDATE;
-- session B (blocks)
START TRANSACTION;
SELECT * FROM trip_seat WHERE trip_id=1 AND seat_no='A1' FOR UPDATE;
-- session C
SELECT * FROM sys.innodb_lock_waits\G
```

Then repeat B with `FOR UPDATE NOWAIT` and `FOR UPDATE SKIP LOCKED`.

L7 — Isolation. Session A: `SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ; START TRANSACTION; SELECT fare FROM trip WHERE id=1;` Session B: update that fare and commit. Session A: re-read → unchanged (snapshot). Now `UPDATE trip SET fare = fare + 1 WHERE id=1; SELECT fare ...` → observe it jumped to B's value + 1. Explain why. Repeat the whole thing at READ COMMITTED.

L8 — Deadlock. Session A locks seat 1 then seat 2; session B locks seat 2 then seat 1. Trigger error 1213, then read `SHOW ENGINE INNODB STATUS\G`.

L9 — Gap locks. At RR, `SELECT * FROM ticket WHERE id BETWEEN 100 AND 200 FOR UPDATE;` in session A, then try `INSERT` with id 150 in session B. Repeat at READ COMMITTED and observe the difference.

L10 — Join algorithms. `EXPLAIN FORMAT=JSON` a join of `ticket` to `trip` with and without an index on `ticket.trip_id`. Identify nested loop vs hash join.

L11 — Anti-join. Write "trips that sold zero tickets" three ways; compare plans.

L12 — Optimistic locking. Implement the version-column update and prove that a stale version affects 0 rows.

14. Interview rapid-fire

1. Difference between 3NF and BCNF, with an example that is 3NF but not BCNF.
2. Why is a clustered index's leaf different from a secondary index's leaf in InnoDB?
3. What is a covering index and how do you confirm you have one?
4. Explain the leftmost prefix rule.
5. Why does `WHERE YEAR(col)=2026` kill an index?
6. Explain MVCC. What is a read view?
7. Difference between REPEATABLE READ in the SQL standard and in InnoDB.
8. What is a next-key lock and what problem does it solve?
9. How does InnoDB detect a deadlock and what does it do?
0. Undo log vs redo log.
1. What does `innodb_flush_log_at_trx_commit=2` risk?
2. `ON` vs `WHERE` in a LEFT JOIN.
3. When would the optimizer choose a hash join over a nested loop?
4. How do you fix N+1 in Eloquent, and how do you detect it?
5. Keyset vs offset pagination.
6. Why is `COUNT(*)` slow on a large InnoDB table?
7. Pick a shard key for X and defend it.
8. Consistent hashing — what problem does it solve?
9. 2PC vs Saga.
0. Why is `STATEMENT` binlog format unsafe?
1. How do you solve read-after-write with async replicas?
2. Semi-sync replication — what exactly does the primary wait for?
3. State CAP correctly. Why is CA not real?
4. PACELC — what does it add?
5. `R + W > N` — what does it guarantee?
6. Optimistic vs pessimistic locking — pick one for seat booking and justify.
7. Why should a transaction never call a payment gateway?
8. What is write skew and which isolation level prevents it?
9. Why does a UUIDv4 PK hurt InnoDB?
0. Partitioning vs sharding.

15. Answers

§1.7

1. Violations: `product_name`/`unit_price` depend only on `product_id` (partial → 2NF); `customer_email` depends on `customer_id` which depends on `order_id` (transitive → 3NF). 3NF: `customer(customer_id, email)`, `product(product_id, name, unit_price)`, `orders(order_id, customer_id)`, `order_item(order_id, product_id, qty, unit_price_at_purchase)`. Keep the historical price on the line item.
2. BCNF requires *every* determinant to be a super key; 3NF exempts FDs whose dependent is a prime attribute. So BCNF ⊂ 3NF.
3. When BCNF/3NF decomposition would lose dependency preservation, or on a hot denormalized read table / warehouse dimension where join cost dominates.
4. `D→A→B→C`, so D determines everything → **D** is the only candidate key. Non-prime B and C depend transitively on D → it's in 2NF (key is single-attribute so no partial FD possible) but **not 3NF**.

§2.7

1. Because a real nested transaction isn't supported by MySQL/PG; Laravel emits `SAVEPOINT trans2` and increments a counter. Only the outermost COMMIT is durable.
2. MySQL implicitly commits each DDL statement, so the schema is left half-migrated — earlier statements are permanent. That's why you write small, individually-safe migrations (or use PostgreSQL, which rolls the whole thing back).
3. The email goes out but the data doesn't exist — an unrollbackable side effect. It also holds locks for the duration of the SMTP call. Dispatch a queued job with `afterCommit()` instead.

§3.4

1. Undo log → atomicity (rollback of uncommitted work); redo log/WAL → durability. Recovery replays redo forward, then applies undo to roll back transactions that were in flight.
2. The commit is written to the OS page cache but fsynced only once per second. A `mysqld` process crash loses nothing; an **OS/machine crash or power loss** can lose up to ~1 second of committed transactions.
3. Because A/I/D are enforced by the engine, whereas C is only as strong as the constraints and application logic you wrote. The engine can't know your business invariants.

§4.11

1. `a=1 AND b=2` ✓ full (two columns); `b=2` ✗; `a=1 AND c=3` ⚠ only `a` used for the seek, `c` filtered afterwards; `a=1 AND b>2 AND c=3` ⚠ `a` and `b` used, the range on `b` stops `c` from being used for the seek.
2. InnoDB's clustered index stores the table physically in PK order, so random UUIDs cause page splits and fragmentation on every insert — and the fat PK is duplicated inside every secondary index leaf. PG's heap has no PK ordering, so only the index itself suffers.
3. `WHERE paid_at >= CURDATE() AND paid_at < CURDATE() + INTERVAL 1 DAY`.
4. Drop `INDEX(user_id)` — it's a redundant leftmost prefix of `(user_id, status)`.

§5.7

1. Full scan of 2.4M rows plus an unindexed sort. Fix: add an index covering the `WHERE` predicate, and extend it to include the `ORDER BY` column (equality columns first, sort column last) so the sort is satisfied by the index.
2. The engine must produce and discard 200 000 rows before returning 20. Fixes: keyset pagination (`WHERE id < :last_id`), or a deferred join on a covering index.
3. When the subquery returns any NULL: `x NOT IN (1, NULL)` evaluates to UNKNOWN for every row, so nothing matches. Use `NOT EXISTS`.
4. Different data volume (plan flips), stale/missing statistics, different indexes between environments, cold buffer pool, concurrent load and lock waits, different server config/version.

§6.5

1. `NOT EXISTS (SELECT 1 FROM ticket k WHERE k.trip_id=t.id)`; `LEFT JOIN ticket k ON k.trip_id=t.id WHERE k.id IS NULL`; `t.id NOT IN (SELECT trip_id FROM ticket)`. Ship `NOT EXISTS` — NULL-safe and usually the best plan.
2. Non-matching rows get NULL on the right side; `NULL = 'x'` is UNKNOWN, so `WHERE` discards exactly the rows the LEFT JOIN was there to preserve.
3. Each trip's expense rows are duplicated once per ticket, so `SUM` is multiplied by the ticket count. Fix: aggregate each side in its own subquery/CTE and join the aggregates, or use `LATERAL`.
4. Hash join — it only handles equi-joins. Merge and nested loop can handle inequality (merge with restrictions).

§7.9

1. Because InnoDB locks *index records*. With no usable index, the scan touches every record and takes a next-key lock on each, so concurrent writers are blocked table-wide.
2. It lets N workers claim disjoint rows from a queue table without blocking each other: `SELECT id FROM jobs WHERE status='pending' ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED;` then mark it running inside the same transaction.
3. Sort the row IDs before locking so both workers acquire in the same order; and/or lock the parent row once instead of three children. Plus retry on 1213.
4. Pessimistic: `lockForUpdate` on the seat row inside a transaction. Optimistic: `UPDATE ... WHERE id=? AND version=?` and treat 0 affected

rows as a conflict. Structural: `UNIQUE(trip_id, seat_no)` on `ticket` and catch the duplicate-key error. Ship the unique constraint **plus** `lockForUpdate` — the constraint is the correctness backstop, the lock gives a clean 409 instead of an exception under contention.

§8.8

1. Two on-call doctors both check "at least one other doctor is on call", both see the other, both go off call → zero on call. Each transaction read a set and wrote a *different* row, so there's no update conflict for RR to detect. Only SERIALIZABLE (or an explicit lock on the whole set) prevents it.
2. Stored value is **40**. The plain `SELECT` used the RR snapshot (100), but `UPDATE` is a locking read that sees the latest committed version (50), so `50 - 10 = 40`. This mismatch is exactly why you shouldn't compute values in app code from a snapshot read.
3. Its read view pins old row versions, so the undo log history list (PG: dead tuples) can't be purged. The tablespace grows and every other reader walks longer version chains.
4. It silently appends `FOR SHARE` to them, taking shared next-key locks — so plain reads now block writers and deadlocks become common.

§9.8

1. `% N` changes for nearly every key when N changes, so almost all data must move. Consistent hashing places shards and keys on a ring, so adding a node only steals the keys in one arc — about 1/N of the data.
2. `operator_id` (or tenant id): high cardinality once you have many operators, reasonably even if you salt/split the largest operator, present in essentially every query (all ticketing queries are scoped to an operator), and stable (a ticket never changes operator). Cross-operator reporting becomes scatter-gather — accept that and push it to an OLAP copy.
3. 2PC gives atomicity across shards but blocks if the coordinator dies and holds locks across the network — poor for user-facing latency. Saga does local transactions (reserve seat → charge wallet) with compensations (release seat) and an outbox for reliable events; eventually consistent but available. For booking, prefer the Saga with a short seat *hold*.
4. A query that can't be routed by the shard key must be sent to every shard and the results merged. p99 becomes the slowest shard's p99, and it degrades as you add shards.

§10.8

1. It replays the SQL text, so anything non-deterministic diverges: `INSERT ... VALUES (UUID())`, `UPDATE ... WHERE ... LIMIT 10` without a deterministic `ORDER BY`, `NOW()` in some contexts, triggers/UDFs.
2. The write went to the primary; the read hit a lagging replica. Fixes: sticky/read-your-writes routing to the primary for that session, read from the primary on write-adjacent endpoints, or wait for the GTID to be applied. Also fix the underlying lag (parallel appliers, shorter transactions).
3. The replica has the events in its relay log but may not have applied them. If the primary dies, the replica still has the data and will apply it — but if the *replica* also dies before applying, or you promote a different replica, those transactions are lost. Semi-sync bounds loss, it doesn't eliminate it.
4. Every replica applies the full write stream, so total write capacity is capped by a single node's apply throughput. Only sharding partitions the write load.

§11.8

1. Because partitions are unavoidable in a network. Choosing "CA" means choosing to be incorrect or to stop working when a partition happens — you haven't avoided the trade-off, you've just declined to decide.
2. `W=4, R=2` satisfies `R+W>N` (6>5) with the cheapest reads. `W=5,R=1` is even faster on reads but the write fails if any node is down.
3. Under a partition where the primary is unreachable: writes stop → **PC**. In normal operation, if you read from async replicas you've chosen **EL** (latency over consistency); if all reads go to the primary, **EC**. Redis used as a cache is PA/EL. So a typical stack is PC/EL — which is exactly why read-after-write bugs appear.
4. AP: a shopping cart, a like counter, session data, an IoT metrics feed — stale or briefly divergent data is harmless and downtime isn't. CP: seat inventory, account balances, distributed locks, unique username registration — serving wrong data is worse than serving an error.